Certified Block Overlap in Subset Sum

Small local dictionaries, bounded search, paid discovery

Jonathan R. Landers

November 2026

Standalone research paper · Working draft

Abstract. An input can have exponentially many distinct subset sums and still admit a short exact search. We partition sorted positive weights into small blocks, represent each block by its distinct sums, and bound how many local choices can remain compatible with any target. The product of these overlap bounds controls the entire search tree. A dynamic program finds the minimum product over contiguous partitions with at most b items per block. If L is binary input length and K_b is that optimum, discovery and exact solving together take $\mathrm{poly}(L)(2^b + K_b)$ bit operations. A separated family has 2^n distinct global sums but $K_3 = 1$; a perturbation condition preserves this property. Full proofs, a reference algorithm, exhaustive finite checks, and two reproducible illustrations make the guarantee explicit. These are structured-input upper bounds, with no claim of general runtime superiority or established literature novelty.

1 The opportunity: distinguish local choices, not all global sums

Subset sum asks whether some selection of positive integer weights totals a specified integer target. There are 2^n selections, but their number alone does not determine the computation needed to answer the question. Repeated sums can be merged; more subtly, sums can remain distinct while their *separation* makes almost every choice impossible.

Consider the blocks $(2, 3, 4)$ and $(20, 30, 40)$. The lower block totals 9. For target 37, the higher block must contribute between 28 and 37; its only sum there is 30. The remaining question is whether the lower block reaches 7. For target 38, the same higher choice survives, but the remainder 8 is unreachable. The interval test therefore excludes whole groups safely, while a surviving interval still requires exact search. We turn this example into a target-uniform bound and charge the work needed to discover a useful partition.

The classical meet-in-the-middle algorithm of Horowitz and Sahni [1] uses $O^*(2^{n/2})$ time; Schroeppel and Shamir [2] retain that time with $O^*(2^{n/4})$ space. Here O^* suppresses factors polynomial in encoded input length. Ordered-sum constructions [3] already show that distinct sums can coexist with efficient decision. Recent worst-case work, including Li and Ye's partial-match approach [4], improves on the classical $1/2$ exponent by testing compatibility of partial representations. Our question is complementary: can a readily discovered local structure certify a short deterministic search on a particular input?

Let $a = (a_1, \ldots, a_n) \in \mathbb{Z}^n_{>0}$, $t \in \mathbb{Z}$, and $[n] = \{1, \ldots, n\}$. The required output is an index set $U \subseteq [n]$ such that $\sum_{i \in U} a_i = t$, or a correct NO answer. Equal values remain distinct indexed items; the empty selection is allowed. Let L be the length of a standard explicit binary encoding of (a, t), including delimiters. Thus $n = O(L)$, and all partial sums and witness masks have $O(L)$ bits. Throughout, $\mathrm{poly}(L)$ denotes a uniform polynomial bound; it hides no additional exponential parameter. Numeric magnitude is a different parameter: Bringmann's randomized $\widetilde{O}(n + t)$ algorithm [5] is near-linear pseudopolynomial time, not polynomial time in the binary length of t. The tilde suppresses logarithmic factors.

2 Local dictionaries and the exact interval test

Sort the items as $w_1 \le \cdots \le w_n$, breaking ties by original index and retaining that index map. Fix $1 \le b \le n$ and cuts $0 = v_0 < v_1 < \cdots < v_m = n$, with $v_j - v_{j-1} \le b$. The jth block is $B_j = \{v_{j-1} + 1, \ldots, v_j\}$. Define

$$S_j = \left\{ \sum_{i \in U} w_i : U \subseteq B_j \right\}, \qquad P_j = \sum_{i=1}^{v_{j-1}} w_i. \tag{1}$$

Store the sorted distinct values in S_j, keeping one original-index witness per value. If higher blocks leave residual target r, a completion using $s \in S_j$ needs a lower contribution $r - s \in [0, P_j]$.

Support compression is an established algorithmic resource. Bringmann, Fischer, and Nakos [6] compute the full truncated support in randomized time $\widetilde{O}(|\mathcal{S}_{\le t}|\sqrt{n})$, where $\mathcal{S}_{\le t}$ comprises all attainable sums at most a nonnegative target t. Here we construct only local supports; the separated family below avoids enumerating its exponentially large global support.

Lemma 1 (Sound exclusion and merging)**.** *Only sums $s \in S_j \cap [r - P_j, r]$ can occur in a completion. Keeping one witness per distinct block sum preserves existence and recovery of one full witness.*

Proof. Every lower selection has sum u with $0 \le u \le P_j$; $s + u = r$ implies $r - P_j \le s \le r$. Conversely, two selections within B_j with equal sum leave the same residual and the same unprocessed blocks. Substituting one for the other preserves a solution and cannot reuse an index, because blocks are disjoint. □

The interval $[s, s + P_j]$ contains every residual target that could use local sum s. Its overlap with other such intervals measures possible branching:

$$\kappa_j = \max_{z \in \mathbb{Z}} |S_j \cap [z - P_j, z]|, \qquad K(\mathcal{B}) = \prod_{j=1}^{m} \kappa_j, \qquad 1 \le \kappa_j \le |S_j| \le 2^b. \tag{2}$$

Here $\mathcal{B} = (B_1, \ldots, B_m)$ is the partition. Intervals are closed: sums exactly P_j apart can both qualify. Figure 1 makes this endpoint issue visible.

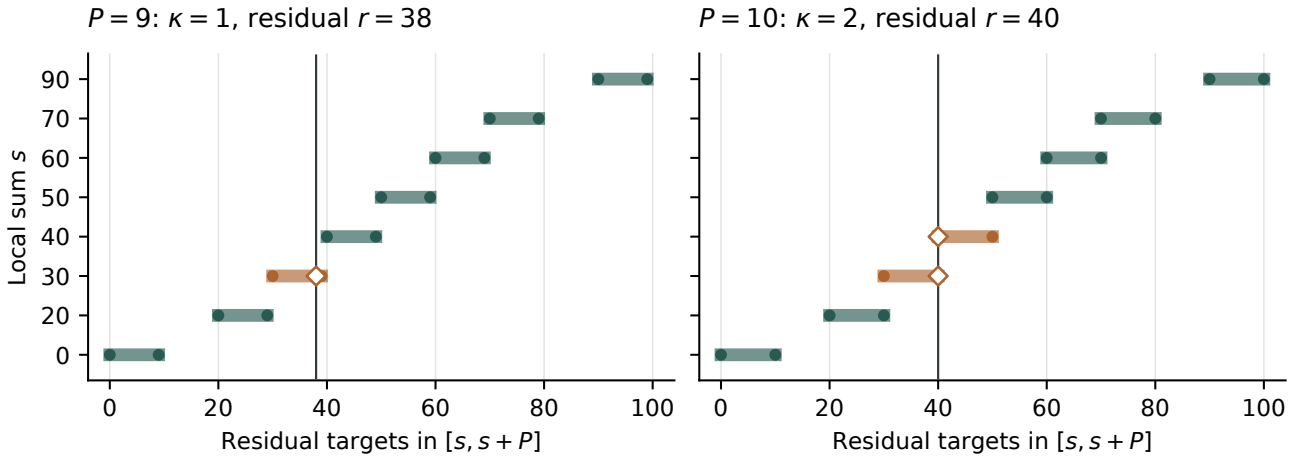


Figure 1: **Overlap is a branching certificate.** Each segment is $[s, s + P]$ for the local sums of $(20, 30, 40)$; orange segments contain the residual target marked by the vertical line and hollow diamonds. With $P = 9$, at most one segment covers any target. With $P = 10$, adjacent segments share an endpoint: residual 40 admits both $s = 30$ and $s = 40$, so $\kappa = 2$. The second panel varies the lower total to illustrate the closed-interval rule; it is not the original two-block input.

3 The overlap product bounds the entire search

Search blocks in descending order. At block j, use two binary searches to locate $S_j \cap [r - P_j, r]$, then recurse on each qualifying sum s with residual $r - s$. At depth m, accept precisely residual zero. Reject $t \notin [0, \sum_i w_i]$ immediately.

Theorem 2 (Fixed-partition search)**.** *The algorithm is exact. Its complete search tree has at most*

$$N(\mathcal{B}, t) \le 1 + \sum_{\ell=1}^{m} \prod_{h=m-\ell+1}^{m} \kappa_h \le (m + 1)K(\mathcal{B}) \tag{3}$$

nodes and at most $K(\mathcal{B})$ complete paths. After construction of the local dictionaries, its bit cost is at most $\mathrm{poly}(L)K(\mathcal{B})$. *An early YES may stop sooner.*

Proof. Induct on the number of remaining blocks. With no blocks, precisely residual zero is attainable. Lemma 1 discards no solution and justifies the stored representative for every surviving local sum. Applying the induction hypothesis to these children proves both YES and NO correctness.

At depth ℓ, each preceding node has at most $\kappa_{m-\ell+1}$ children, independently of its residual. Multiplying these deterministic upper bounds and summing levels gives the first inequality in (3). Since every $\kappa_h \ge 1$, every partial product is at most $K(\mathcal{B})$, proving the second. Binary searches take $O(\log(|S_j| + 1)) = O(b + 1)$ comparisons per internal node. Charge each generated child once; sums, residuals, and masks have $O(L)$ bits. Witness assembly and its final weighted-sum check are polynomial. All these charges fit the stated bit bound. □

No independence of pruning fractions is assumed. The bound includes failed branches, and holds for every target, including NO instances. It need not preserve the number of solutions: equal-sum witnesses have been merged.

4 Finding the best permitted partition, and paying for it

A small product would be of little use if finding its partition required exhaustive search over all cuts. The key observation is that the lower total for a candidate block depends only on where that block begins.

Put $H_v = \sum_{i=1}^{v} w_i$, with $H_0 = 0$. For $1 \le u \le v \le n$ and $v - u + 1 \le b$, let $S_{u,v}$ be the distinct sums of $w_u, \dots, w_v$ and define $\kappa(u, v) = \max_z |S_{u,v} \cap [z - H_{u-1}, z]|$. The exact integer recurrence

$$D(0) = 1, \qquad D(v) = \min_{\max(1, v-b+1) \le u \le v} D(u-1)\kappa(u, v), \qquad K_b = D(n) \tag{4}$$

stores a minimizing predecessor for each v.

Theorem 3 (Optimal contiguous discovery and total cost)**.** *The recurrence finds $K_b = \min_{\mathcal{B}} K(\mathcal{B})$ over all contiguous sorted partitions whose blocks contain at most b items. For that discovered partition, sorting, dictionary construction, optimization, reconstruction, and exact solution together satisfy*

$$\boxed{T(a, t; b) \le \mathrm{poly}(L)(2^b + K_b), \qquad \mathrm{Space}(a, t; b) \le \mathrm{poly}(L)2^b.} \tag{5}$$

The optimum concerns this partition family and this product bound, not minimum actual runtime among all subset-sum algorithms.

Proof. Every partition of the first v items has a last block $[u, v]$. Its final factor $\kappa(u, v)$ uses H_{u-1}, regardless of earlier cuts. Replacing the preceding partition by an optimum cannot increase the product. Induction proves (4) and predecessor reconstruction.

There are at most nb candidate blocks. For a block of length ℓ, start with $(0, \varnothing)$ and repeatedly merge the current sorted sum list with its translate by the next weight, coalescing equal sums and retaining a witness. List sizes are at most 2^h after h items; the total merge work is $O(\sum_{h=1}^{\ell} 2^h) = O(2^\ell)$ arithmetic/comparison operations.

For sorted values $s_1 < \cdots < s_q$, a sliding window moves its left endpoint forward until $s_{\mathrm{right}} - s_{\mathrm{left}} \le H_{u-1}$. The largest window equals $\kappa(u, v)$: any nonempty eligible interval can move its right endpoint to its largest contained support value without losing a contained value. Each endpoint moves at most q times, so scoring takes $O(q)$ comparisons.

Thus discovery costs $O(n \log(n + 1) + nb2^b)$ operations. Discard a candidate list after scoring, store its score, and rebuild only the chosen block lists, adding $O(n2^b)$ operations. Each DP candidate product is at most 2^v, so it uses at most $n + 1$ bits. All other represented integers and witnesses also have polynomial bit length. Theorem 2 then gives (5). More explicitly, schoolbook integer arithmetic permits the conservative bound

$$O\big(L^2 \left[n \log(n + 1) + nb2^b + (m + 1)(b + 1)K_b\right]\big).$$

Store $O(nb)$ scores, predecessors, one temporary candidate list, and the chosen lists, which have at most $m2^b$ records. Depth-first search keeps one path and pending siblings: at most $1 + m(2^b - 1)$ pending nodes. Each record uses polynomially many bits, proving the space bound. □

Reference algorithm.

1. Sort with original indices; compute prefix totals H_v.
2. Construct and score every candidate block of length at most b.
3. Evaluate (4) using exact integer products; reconstruct its cuts and rebuild their witnessed dictionaries.
4. Search downward using Lemma 1; verify any recovered witness against the original input. Exhausting every surviving branch certifies NO.

For $n = 0$, take the empty partition and $K_b = 1$: only $t = 0$ is reachable.

Corollary 4 (An efficiently recognizable tractable class)**.** *Fix integers $c \ge 1$ and $d \ge 0$. For nonempty inputs choose $b = \min\{n, \max\{1, \lfloor c \log_2(L + 1) \rfloor\}\}$. The class $K_b \le (L + 1)^d$ is recognizable and solvable in polynomial bit time, without a supplied partition.*

Proof. Discovery is polynomial because 2^b is polynomial in L; $\lfloor c \log_2(L+1) \rfloor$ is computable as the bit length of $(L+1)^c$ minus one. Compare the computed integer K_b with the threshold and apply Theorem 3 on accepted inputs. □

Other tractable regimes use different structure: Bringmann and Wellnitz [7] give algorithms and conditional lower bounds in a dense numeric regime, while Randolph and Węgrzycki [8] obtain algorithms parameterized by additive doubling $|A + A|/|A|$ of an integer set A. Our κ_j instead measures separation relative to the lower total. These guarantees require their respective hypotheses; none identifies a large overlap product as an intrinsic hardness certificate.

The two costs remain distinct. Increasing b cannot increase K_b, because it permits every earlier partition; nevertheless, constructing larger blocks can cost more. In particular, $b = n$ always permits $K_b = 1$, since the single block has lower total zero. Its dictionary can still require 2^n records. A small search tree alone is not a small total cost. This distinction between stored dictionaries and generated search also motivates worst-case time–space algorithms, including OpenAI's low-space preprint [9]. Its randomized construction and specified word model are separate from the deterministic, input-structural bound here.

5 Exponentially many distinct sums, only one surviving path

For $j = 0, \dots, m-1$, take the sorted triples

$$B_{j+1} = \{2 \cdot 10^j, 3 \cdot 10^j, 4 \cdot 10^j\}, \quad n = 3m, \qquad S_{j+1} = 10^j\{0, 2, 3, 4, 5, 6, 7, 9\}. \tag{6}$$

The total below block $j+1$ is $P_{j+1} = 9\sum_{h<j} 10^h = 10^j - 1$. Every gap between its consecutive support values is at least 10^j, strictly exceeding P_{j+1}. Hence every $\kappa_j = 1$ and $K_3 = 1$. The search visits at most $m+1$ nodes for any target.

Each triple's eight selections produce eight different decimal digits, and no digit carries into the next block. The $8^m = 2^n$ global sums are therefore all distinct. Only $8m$ local records are retained for the triple partition. Because $b = 3$, discovery is polynomial, too: the algorithm never builds the global support.

This family is not itemwise superincreasing: $4 \cdot 10^j < 2 \cdot 10^j + 3 \cdot 10^j$. The maximum-core greedy scan used in the core-and-exceptions approach [10] keeps the first two items of each triple and rejects the third. Indeed, before each triple its retained total is $5(10^j - 1)/9 < 10^j$; the first two items exceed the preceding retained total, while the third does not. That route leaves m exceptions and its enumeration bound is exponential. This compares sufficient bounds, not lower bounds against other algorithms.

Proposition 5 (Irregular scales and perturbations)**.** *Replace the triples by $\{2M_j + e_{j,2}, 3M_j + e_{j,3}, 4M_j + e_{j,4}\}$, where $M_j, E_j \in \mathbb{Z}_{\geq 0}$, $|e_{j,h}| \leq E_j$, and P_j is the actual total of preceding triples. If*

$$M_j - 3E_j > P_j \qquad \text{for every } j, \tag{7}$$

the triples form a contiguous sorted partition with $\kappa_j = 1$, all 2^{3m} global sums are distinct, and $b = 3$ gives polynomial bit time.

Proof. An unperturbed difference between two distinct local subset sums has absolute value at least M_j. Each of the three errors appears in their difference with coefficient in $\{-1, 0, 1\}$, changing it by at most $3E_j$. Thus every local difference remains larger than P_j. Condition (7) also ensures positive weights, increasing order within each triple, and a new smallest weight exceeding the earlier total. The triples are therefore permitted sorted blocks with unit overlap. For two different global selections, their highest differing block contributes a difference larger than the total of all lower blocks; cancellation is impossible. Apply Theorem 3. □

6 Verification, interpretation, and scope

The standard-library reference solver uses exact integers, witnessed sorted merges, a sliding-window overlap calculation, and iterative depth-first search. Its separate checker enumerates subsets directly and independently enumerates every admissible cut set on the exhaustive small domain. The preserved record reports:

Check or constructed-example counter	Count
All sorted multisets of 0–6 items from $\{1, \dots, 5\}$	462
Partition-optimality checks, $b \in \{1, 2, 3\}$	1,386
Additional seeded random inputs, at most 12 items	120
Decision/witness comparisons against exhaustive subsets	37,055
Candidate blocks scored, $n = 300$, $b = 3$	897
Support candidates, including reconstruction	7,966
Search nodes: all-7 YES target / leading-8 NO target	101 / 1

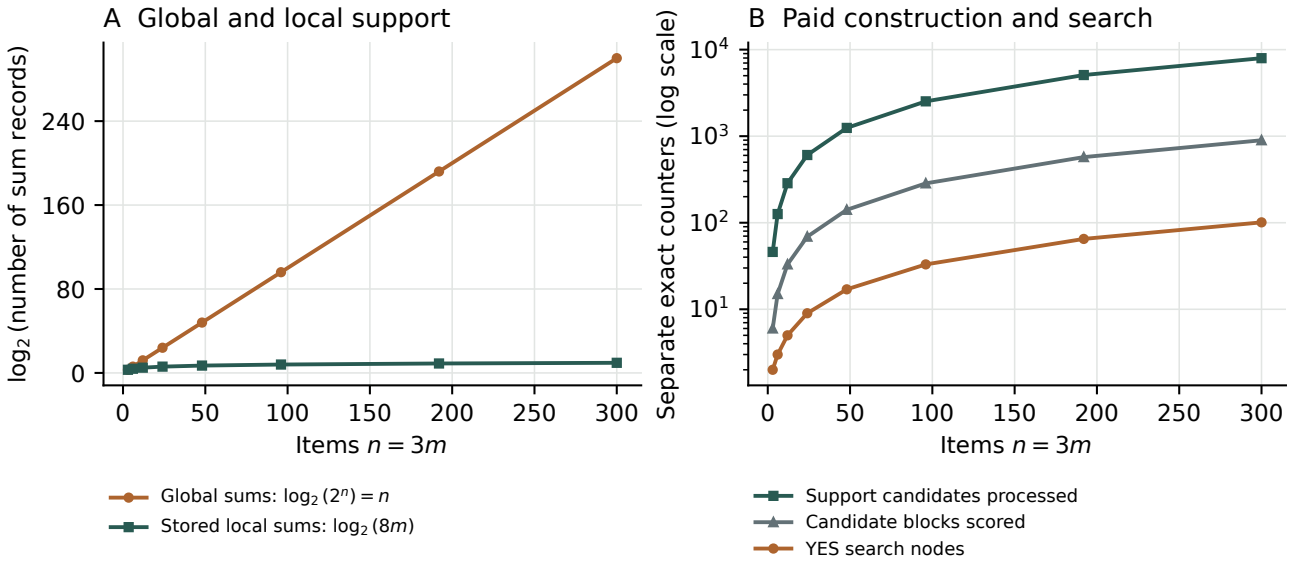


Figure 2: **The separated family, with discovery visible.** For $m \in \{1, 2, 4, 8, 16, 32, 64, 100\}$, panel A compares the proved global support size with the retained local dictionaries; the vertical coordinate is explicitly a base-two logarithm. Panel B shows separate exact counters from the reference implementation with $b = 3$. Support candidates include discovery and rebuilding the chosen lists; the YES target has every decimal digit equal to 7. Counters are neither uniform bit operations nor timings, and are not added together.

Each tested target checks the decision, the original-index witness when present, and (3)'s $(m+1)K$ envelope. Boundary checks cover closed endpoints, duplicate weights, empty inputs, and invalid weights. The seed for the random inputs is 20261104. For the 300-item example, the NO target is $8 \cdot 10^{100-1}$; the global count 2^{300} follows from the proof, not exhaustive enumeration. Global support distinctness is also checked directly on the first one, two, and three triples. These finite checks supplement the proofs; Figure 2 adds deterministic structural illustrations rather than empirical speedup evidence.

The companion source directory contains `solver.py`, `verify.py`, and the original `validation.json`, which records SHA-256 hashes of those sources. From the repository root, run `python notes/01-certified-coverage/block-overlap/verify.py` to repeat the checks, or run the same directory's `build_note.py` to rebuild this paper. The builder checks those hashes, regenerates both plots and evidence-linked numerical macros, writes `paper_evidence.json`, and enforces the six-page limit. Original evidence is preserved.

Relation to distribution shaping. For a fixed partition, $\log_2 K = \sum_j \log_2 \kappa_j$ is an additive description of a multiplicative search bound. Across a specified random input population, this could motivate a statistical model, but no independence, limit theorem, or lognormal law follows from the algebra. Changing an axis cannot change the interval tests or their costs. The exact advantage established here is a discoverable representation with bounded overlap, not a selected density.

Prior work and limits. Singleton blocks with $w_j > P_j$ recover the classical superincreasing case. Earlier ordered-sum constructions already combine distinct subset sums with efficient knapsack decision [3]; that report's abstract establishes relevant precedent, not equivalence with the present overlap-product bound. The contribution here is the self-contained, costed derivation and checked reference implementation. A full novelty audit, a comparative timing study, and stronger partitions beyond contiguous sorted blocks remain open. Large K_b establishes no hardness lower bound; small K_b gives a sufficient guarantee for this exact algorithm.

References

[1] E. Horowitz and S. Sahni. *Computing Partitions with Applications to the Knapsack Problem.* Journal of the ACM 21(2), 277–292, 1974. doi:10.1145/321812.321823.

[2] R. Schroeppel and A. Shamir. *A $T = O(2^{n/2})$, $S = O(2^{n/4})$ Algorithm for Certain NP-Complete Problems.* SIAM Journal on Computing 10(3), 456–464, 1981. doi:10.1137/0210033.

[3] M. D. Atkinson, A. Negro, and N. Santoro. *Sums of Lexicographically Ordered Sets.* Carleton University Technical Report TR-100, May 1987. Institutional abstract.

[4] B. Li and L. Ye. *Subset Sum via Partial Match*. Preprint, version 2, November 2, 2026. arXiv:2610.10446v2.

[5] K. Bringmann. *A Near-Linear Pseudopolynomial Time Algorithm for Subset Sum*. SODA, 2017. arXiv:1610.04712v2.

[6] K. Bringmann, N. Fischer, and V. Nakos. *Beating Bellman's Algorithm for Subset Sum*. SODA, 2025. arXiv:2410.21942v1.

[7] K. Bringmann and P. Wellnitz. *On Near-Linear-Time Algorithms for Dense Subset Sum*. SODA, 2021. arXiv:2010.09096.

[8] T. Randolph and K. Węgrzycki. *Parameterized Algorithms on Integer Sets with Small Doubling: Integer Programming, Subset Sum and k-SUM*. ESA, 96:1–96:19, 2024. doi:10.4230/LIPIcs.ESA.2024.96.

[9] OpenAI. *A Low-Space Algorithm for Worst-Case Subset Sum*. Preprint dated September 26, 2026; fixed repository version.

[10] J. R. Landers. *How Far from Easy? Structural Distance in Subset Sum*. Working draft, 2026. Repository Note V, source and evidence.